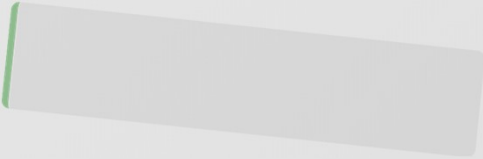




Python

Python Installation and VS Code Setup on Windows

A comprehensive guide to get started with Python development



VS Code

 Windows

 Python

 VS Code

Introduction and Overview

This guide provides a comprehensive overview of installing Python on Windows and setting up Visual Studio Code as an integrated development environment.



Python Installation

- Download Python from the official website
- Install Python with proper PATH configuration
- Verify installation through command line



VS Code Setup

- Install Visual Studio Code editor
- Install Python extension for VS Code
- Select appropriate Python interpreter



Why this matters: Setting up Python with VS Code provides a powerful integrated development environment for writing, running, and debugging Python code.

Downloading Python for Windows

- 1 Visit the Official Website**
Navigate to python.org/downloads
- 2 Select Your Version**
Choose the latest Python 3 release
- 3 Choose Installer**
Select "**Windows installer (64-bit)**"

 **Pro Tip:** Always download from the official Python website to ensure security.



Python Installation Process

Installation Steps



Run the Installer

Double-click the downloaded .exe file to start the Python installer



Add Python to PATH

Check "Add Python.exe to PATH" on the first screen of the installer



Complete Installation

Follow the on-screen prompts and click "Close" when finished

Installation Options



"Install Now" (Default)

- Performs a default installation into your user directory
- Includes standard library, test suite, launcher, and pip
- Typically does not require administrator privileges
- Recommended for most users



"Customize Installation"

- Select specific features to install
- Change the installation location
- Perform an all-users installation (requires admin)
- Install into Program Files directory

Verifying Python Installation

✓ Verification Steps

1 Open Command Prompt

Press Win + R, type "cmd", and press Enter

2 Check Python Version

Type `python --version` or `py -3 --version`

3 Check Pip Installation

Type `pip --version`

> Command Prompt Output

```
C:\Users\Example> python --version
```

```
Python 3.13.0
```

```
C:\Users\Example> pip --version
```

```
pip 23.3.1 from C:\Python313\Lib\site-packages\pip (python  
3.13)
```

Expected Output: If installation was successful, the command prompt will display the

i installed Python version (e.g., Python 3.13.0) and pip version.

Installing Visual Studio Code

 Visual Studio Code is a free, open-source code editor developed by Microsoft for building applications.

1 Download VS Code

Visit code.visualstudio.com and download the Windows installer.

2 Run the Installer

Double-click the downloaded installer and follow the setup wizard.

3 Choose Components

Select components to install. The standard installation includes the core editor and Git version control.

4 Complete Installation

Finish the installation and launch VS Code.

 **Pro Tip:** You can install VS Code for Windows, macOS, and Linux from the same website.

Installing Python Extension in VS Code

1 Open Extensions View

Click the square icon in the Activity Bar on the side, or press

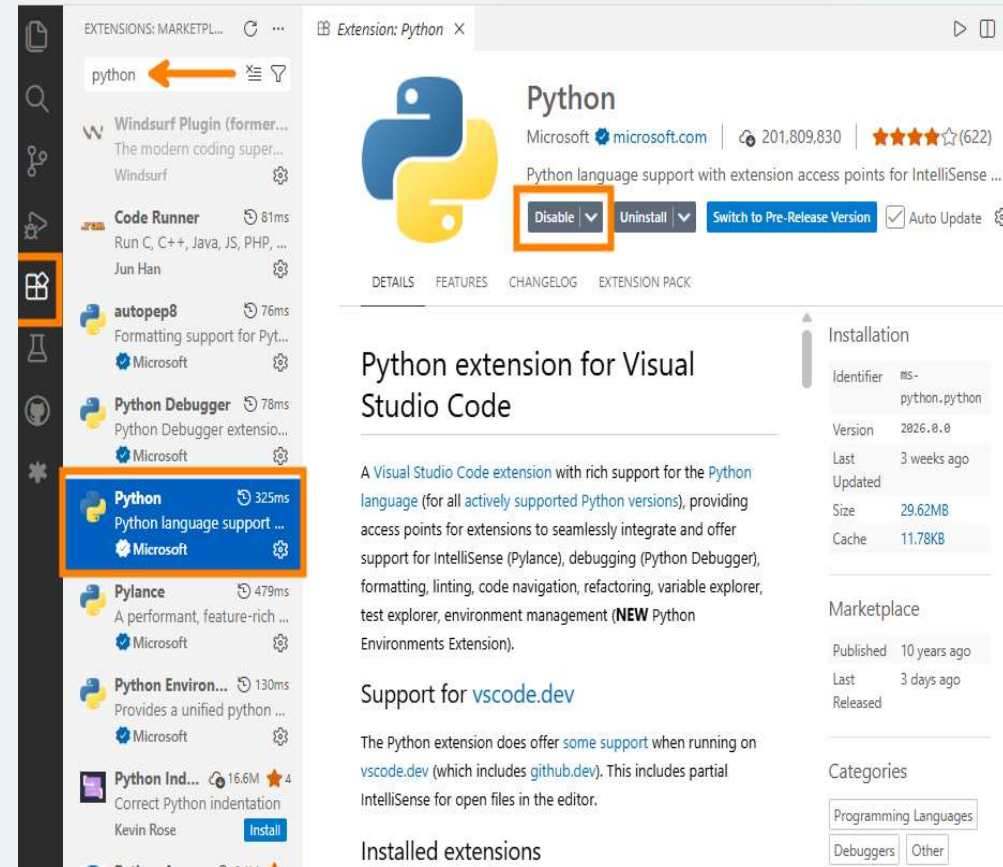
2 Search for Python

Type "Python" in the search bar at the top of the Extensions view

3 Install the Extension

Locate the official "Python" extension by Microsoft and click "Install"

Did you know? The Python extension often automatically bundles other useful extensions such as Pylance (for language server features) and Jupyter (for notebook support).



Selecting Python Interpreter

💡 Why Select an Interpreter?

Selecting the correct Python interpreter ensures VS Code uses the desired version and packages for running, debugging, and providing IntelliSense.

How to Select an Interpreter

- 1 **Open Command Palette:** Press
- 2 **Search for:** Type "Python: Select Interpreter"
- 3 **Choose interpreter:** Select your Python installation from the list

> Command Palette

🔍 Python: Select Interpreter



☑ Select Python Interpreter

✓ Python 3.12.x 64-bit

● Python 3.11.x 64-bit

● Python 3.10.x 64-bit

Creating Virtual Environment

Use a project-specific virtual environment to isolate dependencies and prevent conflicts between different projects.

1 Open Command Palette

Press `Ctrl+Shift+P` to open the Command Palette

2 Search for Command

Type "Python: Create Environment"

3 Select Environment Type

Choose `from the` list of environment types

4 Select Python Interpreter

Choose the Python installation you made earlier

> Command Palette Output

cmd

```
python -m venv myenv
```

cmd

```
myenv\Scripts\activate.bat
```

💡 Why Use Virtual Environments?

- Isolate project dependencies
- Prevent conflicts between projects
- Ensure consistent development environment

i A `.venv` folder will appear in your workspace after creation.

Writing and Running Python Code

1 Create Python File

In File Explorer, select "New File" button and name it with .py extension (e.g., hello.py)

2 Write Code

Enter Python code with IntelliSense auto-completion support

3 Save File

Press Ctrl+S to save your changes

4 Run Code

Click "Run Python File" play button or right-click and select "Run Python File in Terminal"

```
msg = "Hello, World!"  
print(msg)  
# As you type, IntelliSense provides auto-completion
```

 Run Python File

or Press Ctrl+F5

```
Hello, World!
```



Pro Tip: IntelliSense provides auto-completion options for Python modules and methods as you type.

Debugging Python Code

Setting Up Debugging

- Set breakpoints by clicking in the editor's left gutter or pressing **F9**
- Start debugging with **F5** (or Run Python File in Terminal)
- Select **Python File** when prompted

Debug Console

Interact with your program in the Debug Console:

```
> msg.capitalize()
'Hello, World!'
```

Debug Toolbar Controls

```
msg = "Hello, World!"
print(msg)
# Breakpoint here
```

 Continue

Step Over

Step Into



Pro Tip: Use Step Over (F10) to execute lines without entering function calls. Use Step Into (F11) to debug inside functions.

Debug Features and Tools

VS Code provides powerful debugging tools for Python development, enabling comprehensive code inspection and troubleshooting.



Variables Window

- Displays local variables during debugging
- Shows variable names and values
- Allows inspection of data during program execution



Debug Console

- Interact with your program during debugging
- Type expressions at the > prompt
- Evaluate variables and functions



Debug Toolbar

- Continue (F5): Resume program execution
- Step Over (F10): Execute current line
- Step Into (F11): Enter function calls
- Restart (Ctrl+Shift+F5): Restart debugging



Pro Tip: Use the Variables window to inspect your program's state and the Debug Console to test expressions while debugging.

Installing Python Packages



Python Packages

Python's ecosystem thrives on packages from PyPI (Python Package Index) that extend its functionality.



Key Point: Always install packages into your virtual environment to isolate project dependencies.

Installing Packages: Step-by-Step

- 1 **Modify your script** to use the desired package:

```
# hello.py
import numpy as np
msg = "Roll a dice!"
print(msg)
print(np.random.randint(1,9))
```

- 2 **Open integrated terminal** in VS Code:

Terminal → Create New Terminal



- 3 **Install the package** using pip:

```
>_ pip install numpy
```

- 4 **Verify installation** by running your script:

```
>_ python hello.py
```

Managing Dependencies



Recording Dependencies

Python projects rely on external packages from PyPI. It's important to record these dependencies to ensure consistent environments across machines.

- Use **pip freeze** to list all installed packages
- Redirect output to **requirements.txt**
- Commit this file to version control



Best Practice: Update requirements.txt regularly as you install new packages.



Creating requirements.txt

```
# Create requirements.txt file  
pip freeze > requirements.txt
```

bash

```
# Contents of requirements.txt  
numpy==1.21.0  
pandas==1.3.0  
matplotlib==3.4.2
```



Installing from requirements.txt

```
# Install all dependencies  
pip install -r requirements.txt
```

bash

Best Practices Summary

Key recommendations for effective Python development workflow:



Virtual Environments

- Use project-specific virtual environments
- Isolate dependencies to prevent conflicts



Interpreter Selection

- Select correct Python interpreter
- Verify interpreter in Status Bar



Debugging Techniques

- Use breakpoints effectively
- Utilize debug toolbar options



Package Management

- Install packages in virtual environment
- Use requirements.txt for dependencies



Code Organization

- Create clear project structure
- Use meaningful variable names



Extension Usage

- Use Python extension for IntelliSense
- Enable linting for code quality



Key Takeaway: Following these practices ensures efficient Python development.

Next Steps and Resources

Continue your Python journey with these resources and next steps:



Official Documentation

- Python Documentation
- VS Code Documentation



Advanced Features

- Advanced debugging techniques
- Testing functionalities



Web Frameworks

- Django Tutorial
- Flask Documentation



Community Resources

- Python Community
- Stack Overflow

✓ **Your journey continues:** You're now ready to explore advanced Python development with VS Code.